ChatFlow — Learn, Chat, Flow

Software Engineering Project Report

Submitted By:
Group-7
202451129 - Shivang Patel
202451130 - Piyush Satiya
202451131 - Piyush Singh
202451146 - Saksham Bansal

Under the Guidance of:
Dr. Pooja Mishra

Indian Institute of Information Technology, Vadodara

Department of Computer Science

Date: April 7, 2026

Abstract

Contents

Abstract **1**

1 Introduction **4**

1.1 Project Overview 4

1.2 Purpose 4

1.3 Scope 5

1.4 Issues 5

1.4.1 Technical Challenges 5

1.4.2 System Limitations 6

1.4.3 Future Considerations 6

2 Requirement Analysis **7**

2.1 Functional Requirements 7

2.1.1 User Authentication 7

2.1.2 User Profile Management 8

2.1.3 Messaging System 8

2.1.4 Real-Time Communication 8

2.1.5 Online Presence Tracking 8

2.1.6 Unseen Message Tracking 8

2.1.7 Media Sharing 9

2.2 Non-Functional Requirements 9

2.2.1 Performance 9

2.2.2 Security 9

2.2.3 Scalability 9

2.2.4 Usability . . . 9
2.2.5 Reliability . . . 10
2.2.6 Maintainability . . . 10
2.2.7 Availability . . . 10

3 Design and Architecture 11
3.1 System Architecture . . . 11
3.2 Data Flow Diagram (Level 2) . . . 11
3.3 Use Case Diagram . . . 13
3.4 Class Diagram . . . 15
3.5 Module Design . . . 17
3.5.1 Authentication Module . . . 17
3.5.2 Messaging Module . . . 17
3.5.3 Real-Time Communication Module . . . 17
3.5.4 Profile Management Module . . . 17
3.5.5 Media Handling Module . . . 17
3.6 Database Design . . . 17
3.6.1 Entity Relationship Diagram . . . 17
3.6.2 Entities Description . . . 19
3.6.3 Relationships . . . 19

4 Implementation Details 20
4.1 Technologies Used . . . 20
4.1.1 Frontend Technologies . . . 20
4.1.2 Backend Technologies . . . 20
4.1.3 Database and External Services . . . 21
4.2 Feature Implementation . . . 21
4.2.1 Backend Startup, API Binding, and Socket Layer . . . 21
4.2.2 Database Connection and Schema Design . . . 22
4.2.3 Authentication Flow and Protected Middleware . . . 23
4.2.4 Cloudinary Media Integration . . . 24
4.2.5 Messaging Controller Logic . . . 24
4.2.6 Routing and Frontend Context Integration . . . 26

5 Testing and Deployment 27
5.1 Testing . . . 27
5.1.1 Backend Testing . . . 27
5.1.2 Frontend Testing . . . 28
5.1.3 Execution Results . . . 28
5.2 Deployment . . . 28

5.2.1 Environment Configuration 28
5.2.2 Backend Deployment (Vercel) 29
5.2.3 Frontend Deployment (Vercel) 29
5.2.4 Deployment Flow 29

6 Conclusion and Future Work 30
6.1 Conclusion 30
6.2 Future Work 30
6.2.1 Short-Term Improvements 30
6.2.2 Medium-Term Improvements 31
6.2.3 Long-Term Improvements 31
6.3 Final Remarks 31

Chapter 1

Introduction

1.1 Project Overview

ChatFlow is a real-time web-based chat application developed as a full-stack Software Engineering project. The system enables users to communicate instantly through text messages and media sharing using a modern and responsive web interface.

The application is designed using a client-server architecture, where the frontend is built using React and the backend is developed using Node.js and Express. Real-time communication is achieved using WebSocket technology through Socket.io, allowing users to exchange messages without page refresh.

The system also integrates MongoDB as a NoSQL database for persistent data storage and Cloudinary for handling media uploads. ChatFlow provides essential communication features such as user authentication, real-time messaging, online user tracking, and profile management.

1.2 Purpose

The primary purpose of developing ChatFlow is to design and implement a scalable and efficient real-time communication system while applying core concepts of Software Engineering.

This project serves multiple objectives:

- To understand and implement full-stack web development using modern technologies.
- To explore real-time communication using WebSockets.
- To implement secure authentication mechanisms using JSON Web Tokens (JWT).
- To design a system with modular architecture and maintainable code structure.

- To gain practical experience in integrating third-party services such as Cloudinary.

Additionally, this document serves as a reference for understanding system requirements, design decisions, and implementation details for academic evaluation.

1.3 Scope

The scope of ChatFlow is limited to a web-based real-time chat application that supports communication between registered users.

The system provides the following functionalities:

- User registration and login with secure authentication.
- Real-time one-to-one messaging between users.
- Persistent storage of chat messages.
- Online and offline user status tracking.
- Image and media sharing within chat.
- Profile management including profile picture and bio updates.

The application is accessible through modern web browsers and is designed to work across different screen sizes with a responsive interface.

However, the current system does not include advanced features such as group chats, voice/video calling, or end-to-end encryption.

1.4 Issues

Despite achieving the core objectives, certain challenges and limitations were encountered during the development of ChatFlow.

1.4.1 Technical Challenges

- Ensuring real-time message delivery to the correct user without broadcasting to all connected clients.
- Managing synchronization of unseen message counts across different states.
- Handling large image uploads efficiently using base64 encoding.
- Maintaining socket connections and avoiding memory leaks in event listeners.

1.4.2 System Limitations

- The system currently supports only one-to-one messaging and does not include group chat functionality.
- There is no implementation of end-to-end encryption for messages.
- JWT tokens do not have an expiration mechanism in the current version.
- The system relies on continuous internet connectivity.

1.4.3 Future Considerations

These issues highlight areas for improvement and provide direction for future enhancements such as scalability improvements, enhanced security, and additional communication features.

Chapter 2

Requirement Analysis

Requirement analysis is a crucial phase in the Software Development Life Cycle (SDLC) where the needs and expectations of the system are identified and documented. For the ChatFlow application, requirements are categorized into functional and non-functional requirements.

2.1 Functional Requirements

Functional requirements define the core functionalities and operations that the system must perform. These requirements describe how the system behaves in response to user actions.

2.1.1 User Authentication

- The system shall allow users to register by providing their name, email, password, and bio.
- The system shall ensure that each email address is unique.
- The system shall securely store user passwords using hashing techniques.
- The system shall allow registered users to log in using valid credentials.
- The system shall generate and return a JWT token upon successful authentication.
- The system shall restrict access to protected routes for unauthenticated users.
- The system shall allow users to log out and terminate their session.

2.1.2 User Profile Management

- The system shall allow users to view and update their profile information.
- The system shall allow users to upload and update profile pictures.
- The system shall store profile images using cloud-based storage.
- The system shall display user details such as name, bio, and profile picture.

2.1.3 Messaging System

- The system shall support real-time one-to-one messaging between users.
- The system shall store all messages persistently in the database.
- Each message shall include sender, receiver, timestamp, and content.
- The system shall allow users to send text messages and images.
- The system shall display messages in chronological order.

2.1.4 Real-Time Communication

- The system shall establish a real-time connection using WebSockets after login.
- The system shall deliver messages instantly to online users.
- The system shall update message status dynamically without page refresh.
- The system shall notify users about new incoming messages in real time.

2.1.5 Online Presence Tracking

- The system shall maintain a list of currently online users.
- The system shall update user status when they connect or disconnect.
- The system shall display online/offline indicators in the user interface.

2.1.6 Unseen Message Tracking

- The system shall track messages that have not been viewed by the receiver.
- The system shall display the count of unseen messages for each user.
- The system shall mark messages as seen when the conversation is opened.

2.1.7 Media Sharing

- The system shall allow users to upload and send image files in chats.
- The system shall validate file types before uploading.
- The system shall store media files externally and maintain their URLs in the database.

2.2 Non-Functional Requirements

Non-functional requirements define the quality attributes of the system such as performance, security, scalability, and usability.

2.2.1 Performance

The system must ensure efficient and fast message delivery with minimal latency. Real-time communication should be smooth and responsive. The system should handle multiple users simultaneously without degradation in performance.

2.2.2 Security

The system must ensure data security and user privacy. Passwords must be encrypted using secure hashing algorithms. Authentication must be handled using JWT tokens. Unauthorized access to protected resources must be prevented. Sensitive information such as API keys and database credentials must be stored securely.

2.2.3 Scalability

The system should be designed to handle increasing numbers of users and messages. The use of stateless authentication and cloud-based services allows the system to scale horizontally when needed.

2.2.4 Usability

The application must provide an intuitive and user-friendly interface. Navigation should be simple and responsive across different devices. The system should provide feedback to user actions through notifications and visual indicators.

2.2.5 Reliability

The system must ensure that messages are not lost during transmission. Data consistency must be maintained in the database. The system should handle unexpected errors gracefully.

2.2.6 Maintainability

The application should follow a modular design approach, making it easier to update and maintain. Code should be well-structured and follow best practices to allow future enhancements.

2.2.7 Availability

The system should be accessible at all times with minimal downtime. It should depend on stable backend services and database connectivity.

Chapter 3

Design and Architecture

This chapter describes the design aspects of the ChatFlow system, including its architecture, data flow, module structure, and database design. The system is designed using a modular approach to ensure scalability, maintainability, and efficient performance.

3.1 System Architecture

ChatFlow follows a client-server architecture where the frontend and backend communicate through REST APIs and WebSocket connections.

- **Frontend:** Built using React, responsible for handling user interactions and rendering UI components.
- **Backend:** Developed using Node.js and Express, which processes requests and handles business logic.
- **Database:** MongoDB is used to store user data and messages.
- **Media Storage:** Cloudinary is used for storing and serving images.

The system uses HTTP for API communication and WebSocket (Socket.io) for real-time messaging.

3.2 Data Flow Diagram (Level 2)

The Data Flow Diagram (DFD Level 2) provides a detailed representation of how data flows through different processes within the system.

The diagram illustrates internal processes such as authentication, message handling, media upload, and database interactions.

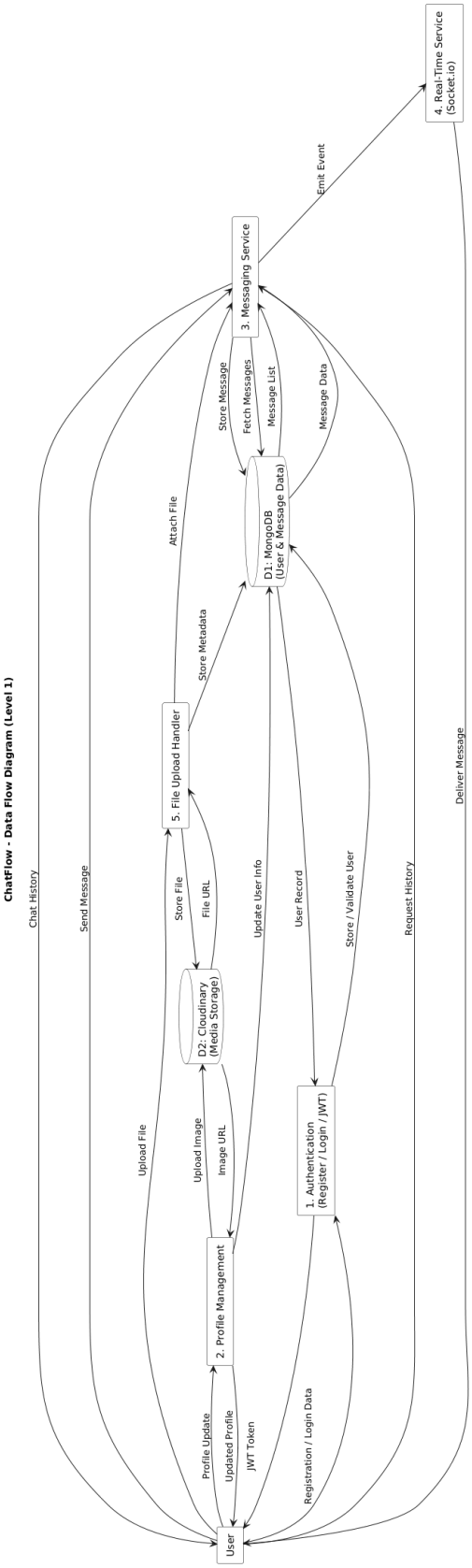

Figure 3.1: Data Flow Diagram (Level 2) of ChatFlow

3.3 Use Case Diagram

The Use Case Diagram represents the interaction between the user and the system functionalities.

The primary actor in the system is the registered user. The user can perform actions such as registering, logging in, sending messages, receiving messages, updating profile, and viewing chat history.

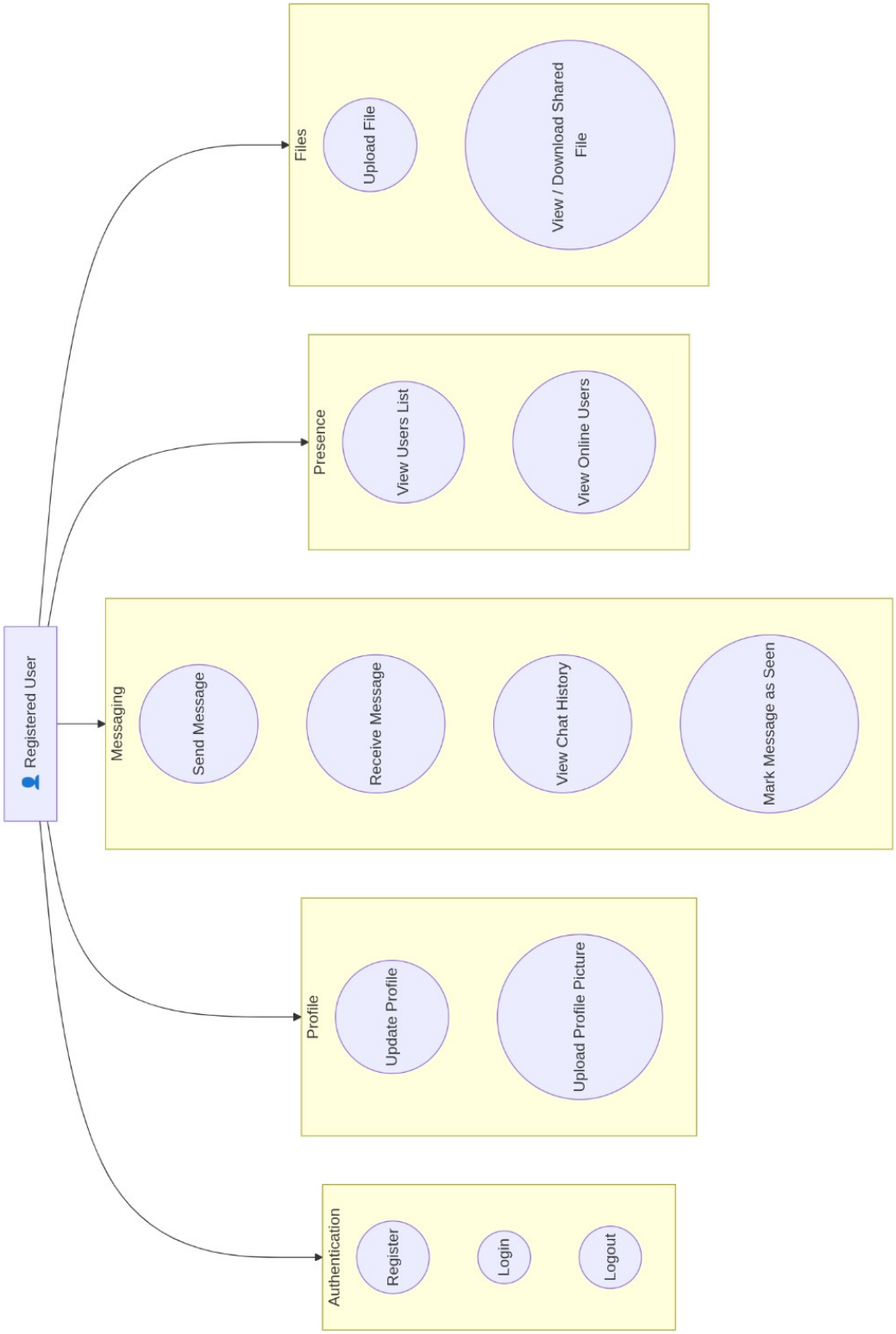

Figure 3.2: Use Case Diagram of ChatFlow

3.4 Class Diagram

The Class Diagram represents the static structure of the system, including classes, attributes, and relationships.

The main classes in the system include:

- **User Class:** Contains attributes such as userId, name, email, password, profile picture, and bio.
- **Message Class:** Contains attributes such as messageId, senderId, receiverId, content, media URL, timestamp, and seen status.

The relationship between these classes defines how users interact through messages in the system.

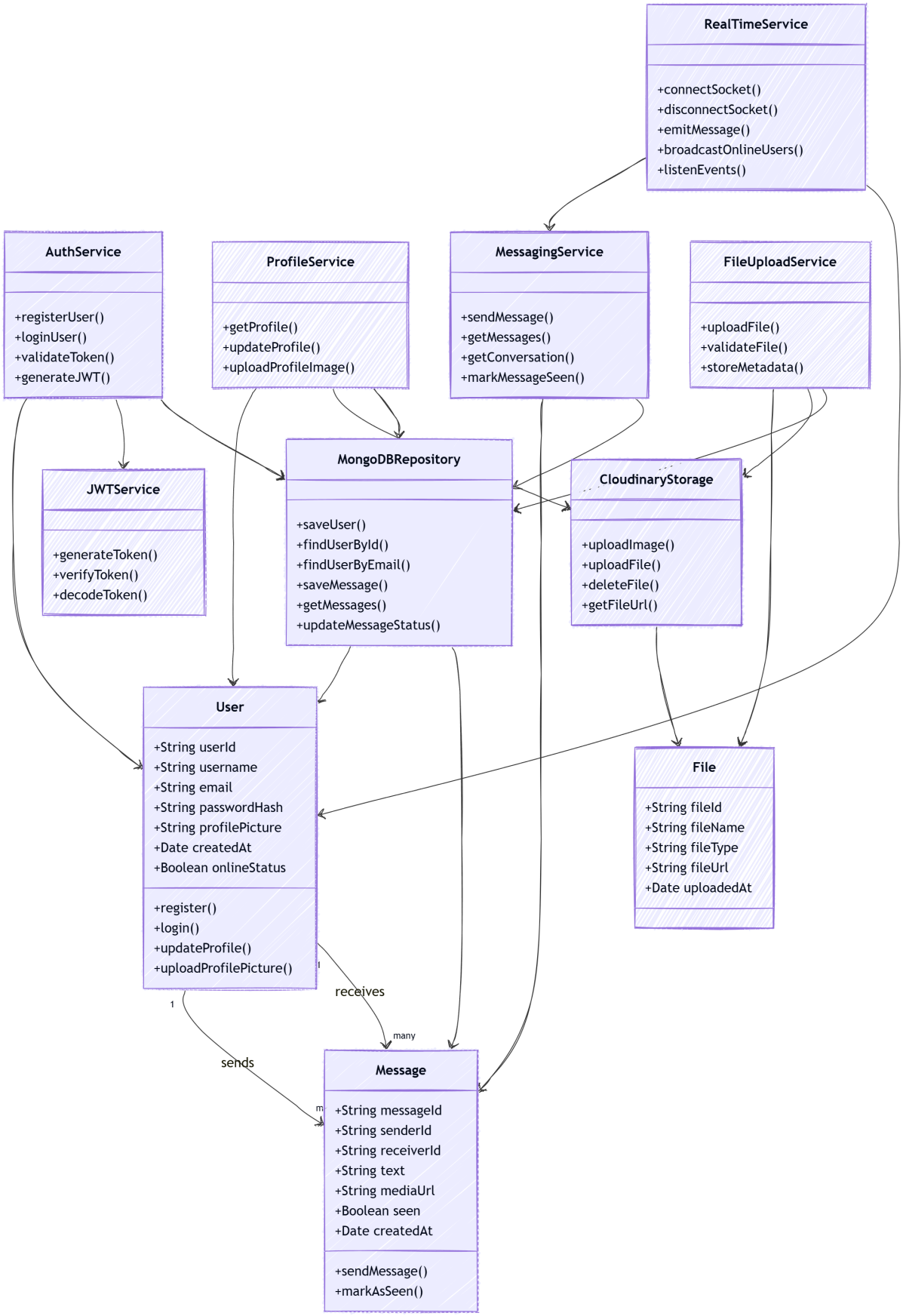

Figure 3.3: Use Case Diagram of ChatFlow

3.5 Module Design

The system is divided into different modules to ensure separation of concerns.

3.5.1 Authentication Module

Handles user registration, login, logout, and authentication using JWT tokens.

3.5.2 Messaging Module

Manages sending, receiving, and storing messages, including real-time delivery.

3.5.3 Real-Time Communication Module

Uses Socket.io to enable instant communication and track online users.

3.5.4 Profile Management Module

Allows users to update personal details such as name, bio, and profile picture.

3.5.5 Media Handling Module

Handles image uploads and integrates with Cloudinary for storage and retrieval.

3.6 Database Design

The system uses MongoDB to manage data in a flexible and scalable way.

3.6.1 Entity Relationship Diagram

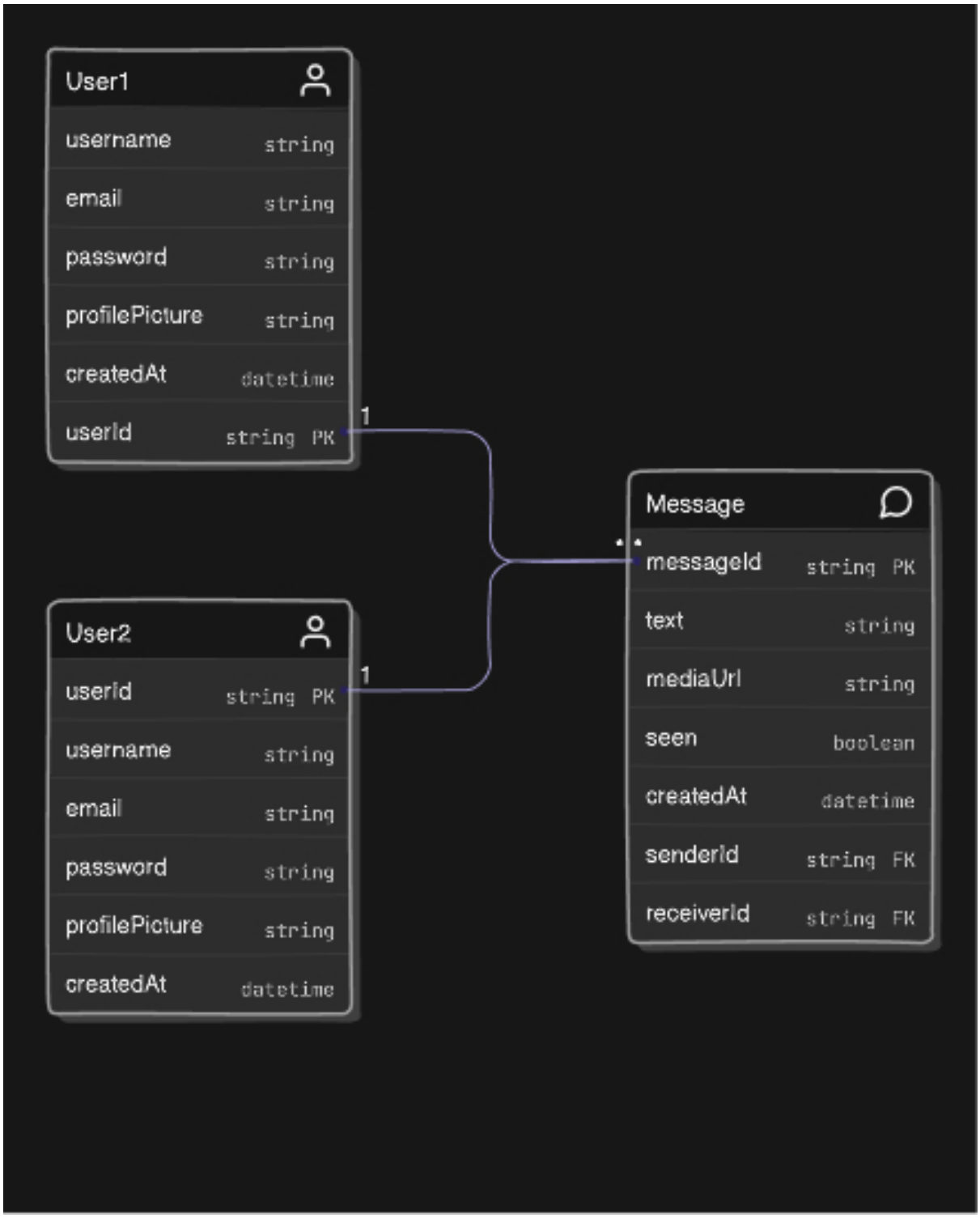

Figure 3.4: Use Case Diagram of ChatFlow

3.6.2 Entities Description

User Entity: Stores user information such as name, email, password, profile picture, and bio.

Message Entity: Stores messages exchanged between users, including sender, receiver, content, media URL, and seen status.

3.6.3 Relationships

- One user can send multiple messages.
- One user can receive multiple messages.
- Each message is associated with one sender and one receiver.

Chapter 4

Implementation Details

4.1 Technologies Used

The ChatFlow application is implemented as a full-stack JavaScript system with clearly separated frontend and backend layers.

4.1.1 Frontend Technologies

- **React (v19):** used for component-based UI development and client-side rendering.
- **React Router:** used for route-level access control between login, home, and profile pages.
- **Axios:** used for HTTP communication with backend APIs.
- **Socket.io Client:** used to subscribe to real-time events such as new messages and online users.
- **Tailwind CSS:** used for responsive styling and modern UI layouts.

4.1.2 Backend Technologies

- **Node.js and Express:** used to implement REST APIs and server-side business logic.
- **Socket.io:** used for bidirectional real-time communication.
- **JWT (jsonwebtoken):** used for stateless authentication.
- **bcryptjs:** used to hash and verify user passwords.

4.1.3 Database and External Services

- **MongoDB with Mongoose:** used for persisting users and messages.
- **Cloudinary:** used to store profile and chat images and return hosted URLs.

4.2 Feature Implementation

This section explains how the main modules were implemented, with direct code references from the ChatFlow repository.

4.2.1 Backend Startup, API Binding, and Socket Layer

The backend initializes Express, creates an HTTP server, mounts API routes, and starts Socket.io for event-driven communication.

```
// server/server.js
const app = express();
const server = http.createServer(app);

export const io = new Server(server, {
    cors: { origin: "*" },
});

app.use("/api/status", (req, res) => res.send("server is live"));
app.use("/api/auth", userRouter);
app.use("/api/messages", messageRouter);
```

The socket layer tracks active users and broadcasts updated presence lists.

```
// server/server.js
export const userSocketMap = {}; // {userId: socketId}

io.on("connection", (socket) => {
    const userId = socket.handshake.query.userId;
    if (userId) userSocketMap[userId] = socket.id;

    io.emit("getOnlineUsers", Object.keys(userSocketMap));

    socket.on("disconnect", () => {
        delete userSocketMap[userId];
        io.emit("getOnlineUsers", Object.keys(userSocketMap));
    });
```

```
});
```

4.2.2 Database Connection and Schema Design

MongoDB connection is handled through a dedicated utility module. The service is initialized before server listen flow.

```
// server/lib/db.js
const connectDB = async () => {
    try {
        const connectionInstance = await mongoose.connect(
            `${process.env.MONGODB_URI}`
        );

        console.log(
            `DB connected | DB host: ${connectionInstance.connection.host}`
        );
    } catch (error) {
        console.log("Error connecting to DB: ", error.message);
        process.exit(1);
    }
};
```

The `User` model defines identity, profile and credential fields:

```
// server/models/User.model.js
const userSchema = new mongoose.Schema(
    {
        email: { type: String, required: true, unique: true, lowercase: true,
    trim: true },
        fullName: { type: String, required: true, trim: true },
        password: { type: String, required: true, trim: true, minlength: 6 },
        profilePic: { type: String, default: "" },
        bio: { type: String, trim: true },
    },
    { timestamps: true }
);
```

The `Message` model stores sender-receiver mapping, optional image payload, and seen flag:

```
// server/models/message.model.js
const messageSchema = new mongoose.Schema(
    {
```

```
        senderId: { type: mongoose.Schema.Types.ObjectId, ref: "User",
    required: true },
        receiverId: { type: mongoose.Schema.Types.ObjectId, ref: "User",
    required: true },
        text: { type: String },
        image: { type: String },
        seen: { type: Boolean, default: false },
    },
    { timestamps: true }
);
```

4.2.3 Authentication Flow and Protected Middleware

Account creation hashes passwords with bcrypt and issues a signed JWT token for stateless sessions.

```
// server/controllers/user.controller.js (signup)
const salt = await bcrypt.genSalt(10);
const hashedPassword = await bcrypt.hash(password, salt);

const newUser = await User.create({
    fullName,
    email,
    password: hashedPassword,
    bio,
});

const token = generateToken(newUser._id);
```

Login verifies credentials by comparing incoming password with the hashed value in database:

```
// server/controllers/user.controller.js (login)
const { email, password } = req.body;
const userData = await User.findOne({ email });

const isPasswordCorrect = await bcrypt.compare(
    password,
    userData.password
);

const token = generateToken(userData._id);
```

Route protection is implemented through JWT verification middleware:

```
// server/middlewares/auth.js
const token = req.headers.token;
const decoded = jwt.verify(token, process.env.JWT_SECRET);
const user = await User.findById(decoded.userId).select("-password");

req.user = user;
next();
```

4.2.4 Cloudinary Media Integration

Cloudinary is configured once at startup and reused in user/message controllers for image uploads.

```
// server/lib/cloudinary.js
cloudinary.config({
    cloud_name: process.env.CLOUDINARY_CLOUD_NAME,
    api_key: process.env.CLOUDINARY_API_KEY,
    api_secret: process.env.CLOUDINARY_API_SECRET,
});
```

Profile image upload flow in controller:

```
// server/controllers/user.controller.js (updateProfile)
const upload = await cloudinary.uploader.upload(profilePic);
updateUser = await User.findByIdAndUpdate(
    userId,
    { profilePic: upload.secure_url, bio, fullName },
    { new: true }
);
```

Chat image upload flow in message controller:

```
// server/controllers/message.controller.js (sendMessage)
let imageUrl;
if (image) {
    const uploadResponse = await cloudinary.uploader.upload(image);
    imageUrl = uploadResponse.secure_url;
}
```

4.2.5 Messaging Controller Logic

The sidebar API computes unseen message counts user-wise using asynchronous queries:

```
// server/controllers/message.controller.js (getUserForSidebar)
const filteredUser = await User.find({ _id: { $ne: userId }
   }).select("-password");
const unseenMessages = {};

await Promise.all(
    filteredUser.map(async (user) => {
        const messages = await Message.find({
            senderId: user._id,
            receiverId: userId,
            seen: false,
        });

        if (messages.length > 0) {
            unseenMessages[user._id] = messages.length;
        }
    })
);
```

Conversation fetch API returns both directions of messages and marks incoming messages as seen:

```
// server/controllers/message.controller.js (getMessages)
const messages = await Message.find({
    $or: [
        { senderId: myId, receiverId: selectedUserId },
        { senderId: selectedUserId, receiverId: myId },
    ],
});

await Message.updateMany(
    { senderId: selectedUserId, receiverId: myId },
    { seen: true }
);
```

Message sending persists payload and emits only to the target receiver socket:

```
// server/controllers/message.controller.js (sendMessage)
const newMessage = await Message.create({
    senderId,
    receiverId,
    text,
    image: imageUrl,
```

```
});

const recieversSocketId = userSocketMap[receiverId];
if (recieversSocketId) {
    io.to(recieversSocketId).emit("newMessage", newMessage);
}
```

4.2.6 Routing and Frontend Context Integration

Backend routes are modular and protected where required:

```
// server/routes/userRoutes.js
userRouter.post("/signup", signup);
userRouter.post("/login", login);
userRouter.put("/update-profile", protectedRoute, updateProfile);
userRouter.get("/check", protectedRoute, checkAuth);
```

```
// server/routes/messageRoutes.js
messageRouter.get("/users", protectedRoute, getUserForSidebar);
messageRouter.get("/:id", protectedRoute, getMessages);
messageRouter.put("/mark/:id", protectedRoute, markMessageAsSeen);
messageRouter.post("/send/:id", protectedRoute, sendMessage);
```

On the frontend, contexts consume these APIs and socket events to keep UI state synchronized in real time.

```
// client/context/ChatContext.jsx
const { data } = await axios.get(`/api/messages/${userId}`);
if (data.success) {
    setMessages(data.messages);
}

socket.on("newMessage", (newMessage) => {
    if (selectedUser && newMessage.senderId === selectedUser._id) {
        newMessage.seen = true;
        setMessages((prevMessage) => [...prevMessage, newMessage]);
        axios.put(`/api/messages/mark/${newMessage._id}`);
    }
});
```

Chapter 5

Testing and Deployment

5.1 Testing

Testing was performed using `Vitest` for both backend and frontend modules. The strategy includes unit testing of controllers/models and integration testing for route plus middleware behavior.

5.1.1 Backend Testing

Backend tests validate:

- user model and message model constraints,
- authentication middleware behavior for valid/invalid JWT,
- controller functions for signup/login/profile update,
- message operations (send, fetch, mark seen),
- route integration using Supertest.

Example integration test from `server/tests/routes.integration.test.js`:

```
const response = await supertest(app)
    .get("/api/auth/check")
    .set("token", "bad-token");

expect(response.body.success).toBe(false);
expect(response.body.message).toContain("jwt verifying error");
```

5.1.2 Frontend Testing

Frontend tests focus on React context logic in authentication and chat modules. Covered scenarios include:

- loading sidebar users and unseen counts,
- fetching message history,
- appending newly sent messages,
- utility function validation.

Example from `client/tests/chat.context.test.jsx`:

```
await ctx.getMessages("u1");
await waitFor(() => expect(ctx.messages).toHaveLength(1));
expect(ctx.messages[0].text).toBe("hi");
```

5.1.3 Execution Results

Test commands were executed successfully on both modules:

- `server`: 7 test files, 21 tests passed.
- `client`: 3 test files, 7 tests passed.

These results indicate that core business logic, API behavior, and context-level state transitions are working as expected.

5.2 Deployment

The project is prepared for deployment with separate frontend and backend configurations.

5.2.1 Environment Configuration

The application relies on environment variables defined in `server/.env.sample` and `client/.env.sample`, including:

- `MONGODB_URI`
- `JWT_SECRET`
- `CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET`
- `VITE_BACKEND_URL`

5.2.2 Backend Deployment (Vercel)

The backend uses a serverless configuration in `server/vercel.json`:

```
{
    "version": 2,
    "builds": [
        {
            "src": "server.js",
            "use": "@vercel/node"
        }
    ],
    "routes": [
        {
            "src": "/(.*)",
            "dest": "server.js"
        }
    ]
}
```

This routes all API requests through `server.js`, where Express routes and Socket.io are initialized.

5.2.3 Frontend Deployment (Vercel)

The frontend is deployed as a SPA with rewrite support in `client/vercel.json`:

```
{
    "rewrites": [
        {
            "source": "/(.*)",
            "destination": "/"
        }
    ]
}
```

This ensures that client-side routes such as `/login` and `/profile` resolve correctly on refresh.

5.2.4 Deployment Flow

In production, the frontend communicates with backend REST endpoints via Axios and establishes Socket.io connection using `VITE_BACKEND_URL`. MongoDB stores user/message data, and Cloudinary serves uploaded media files through secure URLs.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

The ChatFlow application was successfully designed and developed as a real-time web-based chat platform. The project achieved its primary objective of enabling instant communication between users through text and image messaging.

The system integrates modern web technologies such as React for frontend development, Node.js and Express for backend services, MongoDB for data storage, and Socket.io for real-time communication. The use of JWT-based authentication and bcrypt hashing ensures that user data is handled securely.

All major features, including user authentication, real-time messaging, image sharing, online status tracking, and unseen message handling, were implemented successfully. The application provides a smooth and responsive user experience across different devices.

Through this project, practical knowledge of full-stack development, real-time systems, and cloud-based deployment was gained. The modular architecture and use of scalable technologies make the system efficient and maintainable.

6.2 Future Work

Although the ChatFlow system meets its current objectives, several enhancements can be implemented in the future to improve functionality and user experience.

6.2.1 Short-Term Improvements

- Implement JWT token expiration and refresh mechanism for better security.
- Add typing indicators to show when a user is composing a message.
- Enable message deletion functionality.

- Improve input validation and error handling.

6.2.2 Medium-Term Improvements

- Introduce group chat functionality with multiple participants.
- Add emoji support and message reactions.
- Implement message search functionality.
- Support file sharing beyond images (PDF, documents, videos).

6.2.3 Long-Term Improvements

- Implement end-to-end encryption for enhanced privacy.
- Add voice and video calling using WebRTC.
- Integrate push notifications for offline users.
- Develop an admin dashboard for analytics and monitoring.

6.3 Final Remarks

The ChatFlow project demonstrates the practical implementation of a scalable and real-time communication system. With further enhancements, it has the potential to evolve into a full-featured messaging platform comparable to modern chat applications.